

Building AI Agents That Actually Work

A builder's field guide to Claude, Gemini AI Studio, and on-chain agent commerce

Anaydike Divine (@sage_senpeak) | 2026

Who this is for

You already know what an API is.

You have probably called a language model at least once.

But when someone says "build an agent," you want to know what that actually means in code: the real structure, the failure modes, the specific API calls.

That is this guide.

I wrote it after building Elfgents on the CROO Agent Protocol: a callable, paid verification service that other agents hire per job. Three services. Real on-chain payments. Tamper-proof receipts. Everything I cover here has run in production.

Nothing is hypothetical.

Part 1: What an Agent Actually Is

The definition that matters

A chatbot responds to what you type.

An agent takes a goal and figures out the steps.

The difference is a loop.

```
sense → plan → act → observe → repeat
```

- **Sense:** Read the current environment. What state are you in? What has already happened?
- **Plan:** Decide what to do next. Which tool? Which action? What order?
- **Act:** Execute it. Call the tool. Write the file. Hit the API.
- **Observe:** Read the result. Did it work? What changed? What do you know now?
- **Repeat** until the goal is done, or until you hit a defined stopping condition.

This is not a metaphor. It is the literal code structure you write.

The model generates text.

You parse that text for tool calls.

You execute the tool.

You append the result to the conversation.

The model generates again.

That is the agent loop. Every framework, every SDK, every "agentic system" is a wrapper around this pattern.

Three kinds of tasks (choose honestly)

Before picking an API or a framework, know what you are actually building.

Single call. You need one answer. Summarize this document. Classify this email. Extract these fields. One request, one response. This is not an agent. Do not add a loop.

Workflow. You know the steps in advance. Step 1 calls the data API. Step 2 runs the model. Step 3 writes the output. You write the orchestration in code. The model handles each step, not the routing. This is a scripted pipeline, not an agent.

Agent. You do not know the steps in advance. The model decides what to do next based on what it just learned. You give it a goal, some tools, and measurable stopping conditions. It figures out the path.

Most things people call agents are workflows.

That is fine. A reliable workflow beats a hallucinating agent every time. Know which one you need before you write a line of code.

When to actually build an agent

Ask four questions before reaching for the agent tier:

- 1. Is the task multi-step and hard to fully specify in advance?
- 2. Does the outcome justify higher cost and latency?
- 3. Is the model capable at this specific task type?
- 4. Can errors be caught and recovered from? (tests, rollbacks, review gates)

If the answer is no to any of those, stay at the workflow tier.

Agents are expensive. They are slow. They can get stuck in loops. They are the right tool when the problem is genuinely open-ended and the value of autonomous exploration outweighs the risk of failure.

Part 2: Claude API

Model selection

Current production models as of mid-2026:

Model	ID	Context	Input	Output
Claude Opus 4.8	claude-opus-4-8	1M	\$5/MTok	\$25/MTok
Claude Sonnet 4.6	claude-sonnet-4-6	1M	\$3/MTok	\$15/MTok
Claude Haiku 4.5	claude-haiku-4-5	200K	\$1/MTok	\$5/MTok
Claude Fable 5	claude-fable-5	1M	\$10/MTok	\$50/MTok

Default to Opus 4.8. It is the right balance of capability and cost for agent work. Downgrade to Sonnet 4.6 only when you have a specific reason (routing, classification, cheap preprocessing). Use Haiku for tasks that are simple and high-volume.

Fable 5 is Anthropic's most capable model. It has different API behavior from the Opus family (covered below). Use it when Opus 4.8 is not enough.

Setup

```
npm install @anthropic-ai/sdk
```

```
import Anthropic from "@anthropic-ai/sdk";

const client = new Anthropic({
  apiKey: process.env.ANTHROPIC_API_KEY,
});
```

Your API key lives in `.env`. Never hardcode it. Never commit it. That covers 90% of the security advice you will read elsewhere.

Your first real call

```
const response = await client.messages.create({
  model: "claude-opus-4-8",
  max_tokens: 1024,
  messages: [
    { role: "user", content: "What is the capital of France?" }
  ]
});

console.log(response.content[0].text);
```

That is a single call. No loop. One question, one answer.

The response shape you need to know:

```
response.content[0].type // "text" or "tool_use"

response.content[0].text // the model's reply (when type is "text")

response.stop_reason      // "end_turn", "tool_use", "max_tokens", "refusal"
```

Always check `stopreason` before reading `content`. On Fable 5, a refusal returns `stop reason: "refusal"` with an empty content array. Do not try to read text that is not there.

Adaptive thinking

Opus 4.6 and later use adaptive thinking. This is how you enable it:

```
const response = await client.messages.create({

  model: "claude-opus-4-8",

  max_tokens: 8000,

  thinking: { type: "adaptive" },

  messages: [...]

});
```

`budgettokens` is deprecated on Opus 4.7 and 4.8. On Fable 5, thinking is always on: you omit the parameter entirely or pass `{type: "adaptive"}`. Sending `{type: "enabled", budget tokens: N}` on these models returns a 400 error.

Use adaptive thinking for anything that requires multi-step reasoning. It is not a feature you turn on for marketing reasons. It is the mechanism that lets the model think before it answers.

Streaming

If your `max_tokens` is above 16K, you need streaming. A synchronous request that large will time out before the model finishes.

```
const stream = await client.messages.stream({
  model: "claude-opus-4-8",
  max_tokens: 32000,
  thinking: { type: "adaptive" },
  messages: [...]
});

const final = await stream.finalMessage();
```

`.finalMessage()` collects the full response from the stream. Use it when you do not need to process individual chunks.

Use streaming by default for any agent loop. Agent responses are long. Tool results add up. Do not assume the response fits in a non-streaming call.

Fable 5 specifics

Fable 5 has different behavior from the Opus family. The key differences:

Thinking is always on. You cannot disable it. Omit the parameter or pass `{type: "adaptive"}`.

No sampling parameters. `temperature`, `topp`, and `top k` are not accepted on Fable 5 or Opus 4.7/4.8. Remove them before migrating.

Refusals are explicit. A safety refusal returns `stop_reason: "refusal"` with an empty content array. If you use Fable 5 in production, add server-side fallbacks:

```
const response = await client.messages.create({
  model: "claude-fable-5",
  max_tokens: 8000,
  betas: ["server-side-fallback-2026-06-01"],
  fallbacks: [{ model: "claude-opus-4-8" }],
  messages: [...]
});
```

With fallbacks enabled, a declined request is transparently re-served by Opus 4.8 in the same call. You do not need to retry manually.

No assistant prefill. On Opus 4.6 and all newer models, you cannot add an `assistant` turn at the end of the messages array to steer the response. It returns a 400. Remove any prefill logic before upgrading.

Part 3: Tool Use

What tools are

Tools are how you give an agent hands.

Without tools, a model can only read and write text. It cannot call your database. It cannot search GitHub. It cannot write a file. It cannot check whether a claim is supported by its sources.

A tool definition has three parts:

```
{
  name: "fetch_source",
  description: "Fetch a URL and return its text content. Use this to read the content of a source
before judging it.",
  input_schema: {
    type: "object",
    properties: {
      url: {
        type: "string",
        description: "The URL to fetch"
      }
    },
    required: ["url"]
  }
}
```

The model reads the description to decide when to call it.

The `input_schema` tells the model what to send.

The better your description, the better the model decides when and how to use the tool.

The manual tool loop

This is what every framework hides under the hood:


```

async function agentLoop(
  client: Anthropic,
  tools: Tool[],
  initialMessages: Message[]
) {
  let messages = [...initialMessages];

  while (true) {
    const response = await client.messages.create({
      model: "claude-opus-4-8",
      max_tokens: 8000,
      thinking: { type: "adaptive" },
      tools,
      messages,
    });

    // Model is done
    if (response.stop_reason === "end_turn") {
      return response.content;
    }

    // Model wants to use a tool
    if (response.stop_reason === "tool_use") {
      // Append the model's response (including the tool call)
      messages.push({ role: "assistant", content: response.content });

      // Execute each tool call and collect results
      const toolResults = [];
      for (const block of response.content) {
        if (block.type === "tool_use") {
          const result = await executeTool(block.name, block.input);
          toolResults.push({
            type: "tool_result",
            tool_use_id: block.id,
            content: JSON.stringify(result),
          });
        }
      }
    }
  }
}

```

```
        });  
    }  
}  
  
// Append the tool results and loop again  
messages.push({ role: "user", content: toolResults });  
  
continue;  
}  
  
// Any other stop reason: handle it and break  
break;  
}  
}
```

That is the complete pattern. Every agent framework is a variation of this code.

The important details:

1. You append the model's full response (including tool calls) before appending tool results.
2. Tool results go in a `user` turn, not an `assistant` turn.
3. Every tool call has a `tooluseid`. The result must match it.
4. You loop until `stopreason` is `end turn`.

The SDK tool runner (beta)

If you want to skip the manual loop, the SDK has a beta tool runner:

```

import { betaZodTool } from "@anthropic-ai/sdk/beta";
import { z } from "zod";

const fetchSource = betaZodTool({
  name: "fetch_source",
  description: "Fetch a URL and return its text content",
  schema: z.object({ url: z.string() }),
  handler: async ({ url }) => {
    const res = await fetch(url);
    return { text: await res.text() };
  },
});

const response = await client.beta.messages.runTools({
  model: "claude-opus-4-8",
  max_tokens: 8000,
  tools: [fetchSource],
  messages: [{ role: "user", content: "Check if this source supports the claim..." }],
});

```

The tool runner handles the loop automatically. You define tools with Zod schemas and handlers. The SDK calls the right handler, appends results, and loops until `end_turn`.

Use the manual loop when you need approval gates, custom logging, or conditional execution between tool calls. Use the tool runner when you just want things to work.

Tool design principles

One job per tool. A tool that does three things is a tool the model will misuse.

Describe the output, not just the input. Tell the model what it will get back. If the output can be null or an error, say that.

Keep schemas flat. Deep nested objects in input schemas confuse the model. Flatten where you can.

Return structured data. JSON is better than prose. The model will parse your tool result in context. Make it easy.

Fail loudly. A tool that silently swallows an error and returns an empty result will cause the model to hallucinate a follow-up step. Return the error string in the content field. Let the model see it.

Part 4: Gemini AI Studio

What it is

Gemini AI Studio is Google's playground for the Gemini model family.

It is the fastest way to prototype a Gemini-powered application.

You get a web interface, a direct API key, and a code export button that shows you the request in your language.

The production API is Google's Generative Language API, accessed through the `@google/generative-ai` SDK.

```
npm install @google/generative-ai
```

```
import { GoogleGenerativeAI } from "@google/generative-ai";

const genAI = new GoogleGenerativeAI(process.env.GEMINI_API_KEY);

const model = genAI.getGenerativeModel({ model: "gemini-2.0-flash" });
```

Model options

The two models you will use most:

Gemini 2.0 Flash. Fast and cheap. Good for workflows, classification, and preprocessing. This is the default for high-volume work.

Gemini 1.5 Pro / 2.0 Pro. Longer context, better reasoning. Use when you need to process large documents or when Flash is not good enough.

Gemini AI Studio gives you a free tier. It has rate limits. Moving to production means moving to the Google Cloud API with billing enabled.

Function calling in Gemini

Gemini's equivalent of tool use is function calling. The structure is similar to Claude's tools:

```
const tools = [
  {
    functionDeclarations: [
      {
        name: "search_github",
        description: "Search GitHub for repositories matching a query",
        parameters: {
          type: "object",
          properties: {
            query: {
              type: "string",
              description: "The search query",
            },
          },
          required: ["query"],
        },
      },
    ],
  },
],

const result = await model.generateContent({
  contents: [{ role: "user", parts: [{ text: "Find Go agent frameworks" }] }],
  tools,
});
```

When the model wants to call a function, `result.response.functionCalls()` returns an array. Execute each call, then feed the results back:

```
const functionCalls = result.response.functionCalls();
if (functionCalls) {
  const functionResponses = [];
  for (const call of functionCalls) {
    const output = await executeFunction(call.name, call.args);
    functionResponses.push({
      functionResponse: {
        name: call.name,
        response: output,
      },
    });
  }
  // Continue the conversation with function results
  const followUp = await model.generateContent({
    contents: [
      ...,
      { role: "model", parts: [{ functionCall: functionCalls[0] }] },
      { role: "user", parts: functionResponses },
    ],
    tools,
  });
}
```

The pattern is the same as Claude: append the model's response, append your results, continue.

Gemini vs Claude: when to use which

Use Claude when:

- You need the strongest reasoning model available (`claude-opus-4-8` , `claude-fable-5`)
- You need adaptive thinking for multi-step agent work
- You are already in the Anthropic ecosystem

Use Gemini when:

- You want the Google Search grounding feature (native web search, no API key required beyond Gemini)
- You are building on Google Cloud and want native integration
- You need a free-tier model for development and prototyping
- You are running high-volume preprocessing and Flash's pricing fits better

These are not mutually exclusive. I have run architectures where Gemini Flash handles document preprocessing and Claude Opus handles the reasoning step. Pick the right model for each job.

Part 5: Problems You Will Actually Hit

This section is the reason I wrote this guide.

Every tutorial shows the happy path. What follows are the problems that show up on Tuesday when you thought you were done.

Context explosion

The context window fills up.

In a long agent loop, every tool result you append grows the prompt. After enough turns, you are sending 50K tokens per request. The model starts losing track of early context. Latency climbs. Cost climbs.

Fix: Summarize completed steps before they go stale. Keep the recent N turns in full. Compress everything older into a running summary. This is called context management and it is not optional for long-running agents.

Tool call hallucinations

The model calls a tool with arguments that look plausible but are wrong.

A URL that does not exist. A field name that differs from your schema. A parameter that should be a number but arrives as a string.

Fix: Validate tool inputs before you execute them. Return a structured error from your tool handler if the input is malformed. Feed that error back into the loop. Let the model self-correct.

Also: use `strict: true` on your tool schemas when the SDK supports it. It tightens the model's output against your schema.

Infinite loops

The model gets stuck.

It keeps calling the same tool with slightly different arguments because no stopping condition is met. Or the tool keeps failing and the model keeps retrying with no progress.

Fix: Set a hard max on loop iterations. Ten steps is usually generous. Track which tools have been called in each session. If the same tool is called three times with identical inputs, break and return an error.

```
let steps = 0;
const MAX_STEPS = 10;

while (steps < MAX_STEPS) {
  steps++;
  // ... agent loop body
}
```

A loop that runs forever is worse than an agent that gives up.

Rate limits

You will hit them. Especially if you are running multiple agent sessions in parallel.

Claude's rate limits are per-API-key and are based on tokens per minute (TPM) and requests per minute (RPM). Limits vary by model and tier.

Fix: Implement exponential backoff. The SDK does not retry for you by default.


```
async function callWithRetry(fn: () => Promise<any>, maxRetries = 3) {  
  for (let i = 0; i < maxRetries; i++) {  
    try {  
      return await fn();  
    } catch (error: any) {  
      if (error.status === 429) {  
        const wait = Math.pow(2, i) * 1000;  
        await new Promise((resolve) => setTimeout(resolve, wait));  
        continue;  
      }  
      throw error;  
    }  
  }  
}
```

For high-volume work, look at Anthropic's Message Batches API. It processes requests asynchronously at lower cost. Not for real-time agent loops, but good for batch classification or preprocessing.

Thinking token cost

Adaptive thinking is not free.

When you enable `thinking: {type: "adaptive"}`, the model generates reasoning tokens before its response. These count toward your input token cost on the next turn when you replay the thinking blocks.

For simple tasks, adaptive thinking is overkill and expensive. For multi-step reasoning, it is essential.

Fix: Do not enable adaptive thinking by default on every call. Use it on the planning step and the complex reasoning steps. Use a cheaper call (or skip thinking entirely) for tool-result interpretation when the task is straightforward.

Cold start on environment setup

Your agent assumes dependencies are installed, services are running, or keys are available. On a cold environment, those assumptions fail silently or with cryptic errors.

Fix: Validate your environment before starting the agent loop. Check that required environment variables exist. Ping required services. Return a clear error if the environment is not ready.

```
function validateEnvironment() {  
  
  const required = ["ANTHROPIC_API_KEY", "GITHUB_TOKEN"];  
  
  for (const key of required) {  
  
    if (!process.env[key]) {  
  
      throw new Error(`Missing required environment variable: ${key}`);  
  
    }  
  
  }  
  
}
```

This saves you from spending 40 minutes debugging a tool failure that turns out to be a missing API key.

Tool output that the model cannot parse

You return a large JSON blob from a tool. The model tries to reason about it. It picks the wrong field. Or it quotes back the raw JSON in its text response instead of synthesizing from it.

Fix: Keep tool outputs small and focused. Return only what the model needs for its next decision. If the raw output is large, summarize it in the handler before returning.

```
// Bad: return the entire GitHub API response (300+ fields)
return githubRepo;

// Good: return only what the model needs
return {
  name: repo.name,
  stars: repo.stargazers_count,
  description: repo.description,
  lastPushed: repo.pushed_at,
  topics: repo.topics,
};
```

Prompt injection in tool results

Your tool fetches external content: a web page, a document, a user-provided string.

That content can contain instructions that try to override your system prompt. "Ignore all previous instructions and..." is the classic example. It works more often than people admit.

Fix: Wrap external content in a clear boundary in your tool result:

```
--- BEGIN FETCHED CONTENT ---

(content here)

--- END FETCHED CONTENT ---
```

Then instruct the model in your system prompt to treat anything between those markers as data, not instructions. Not a perfect defense, but it reduces the surface area significantly.

Part 6: Managed Agents

When you want Anthropic to run the agent loop and host the execution environment, use Managed Agents.

The difference from the manual loop: Anthropic hosts the container where Claude runs tools. File operations, bash, code execution all run server-side. You create an agent once, then create sessions against it.

```
// Create the agent once. Store the returned ID.

const agent = await client.beta.agents.create({

  model: "claude-opus-4-8",

  name: "research-agent",

  description: "Searches and summarizes technical topics",

  system: "You are a research assistant...",

  tools: [{ type: "bash_20250124" }],

});


// Create a session per task. Do not create a new agent each time.

const session = await client.beta.agents.sessions.create(agent.id, {

  messages: [{ role: "user", content: "Summarize the latest CAP SDK changes" }],

});
```

The session streams events as the agent works. You listen for completion or error.

Use Managed Agents when:

- The task needs a persistent workspace (file system, installed packages)
- You want Anthropic to handle retry, recovery, and loop management
- You need a stateful, versioned agent configuration used across many sessions

Stick to the manual loop when:

- You need to approve or modify tool calls before they execute
 - You are running your own tool infrastructure
 - You need custom logic between steps
-

Part 7: Building a Callable, Paid Agent

What CROO/CAP is

The CROO Agent Protocol (CAP) is a standard for agent commerce.

Instead of an agent being a private service inside your codebase, CAP lets you register it as a public, callable service. Other agents find it in the Agent Store, negotiate a price, pay in USDC, and receive the output.

Every completed job generates a tamper-proof receipt on-chain. The hiring agent can attach that receipt to its own delivery as proof of work.

This is not a toy. It is a coordination layer for a world where agents hire other agents to get things done.

How Elfgents works

Elfgents is the agent I built on CAP. It lists three callable services:

verify handles claim verification. A caller sends a claim and a list of sources. Elfgents fetches each source, judges whether the sources actually support the claim, and returns a signed receipt with a verdict, a confidence score, and per-source citations. With an Anthropic API key, a model does the grading. Without one, a deterministic keyword-overlap heuristic runs instead. The result is reproducible either way.

recon handles prior-art research. A caller sends a hackathon brief, a track, or a topic. Elfgents searches GitHub for existing projects, ranks them by relevance and recency, reads their READMEs, and synthesizes an angle. The output is a ranked list of repos plus a strategy paragraph that explains what gap still exists. Requires a GitHub token for the live API calls.

validate handles schema conformance. A caller sends a deliverable and a JSON Schema. Elfgents checks whether the payload matches the schema and returns a receipt with validation results. No API key needed. No model call. Deterministic.

All three return the same receipt shape: content-addressed with `keccak256`, hash-chained to the previous job in the session, signed by the agent's wallet. One service or three, the audit trail is one tamper-evident sequence.

The CAP order lifecycle

Every paid job on CAP follows this path:

```
negotiate → accept → order created → pay → paid → do the work → deliver → settle
```

1. **Negotiate.** A buyer agent finds your service in the Agent Store and sends a negotiation request at your listed price.
2. **Accept.** Your listener catches the `NegotiationCreated` event and calls `acceptNegotiation(id)`. This creates the on-chain order.
3. **Pay.** The buyer calls `payOrder`. USDC goes into a CAPVault escrow. You do not touch it yet.
4. **OrderPaid.** Your listener catches this event. You read the buyer's requirements from the order. You run your service.
5. **Deliver.** You call `deliverOrder` with your result as a Schema deliverable. The receipt goes to the buyer.
6. **Settle.** On delivery, the escrow releases. The platform takes a fee. The remainder goes to your agent's AA wallet.

The AA wallet is an ERC-4337 / ERC-6551 account. It is not a personal wallet. It is the agent's wallet. It accumulates earnings from every job it completes.

The agent's identity is a DID backed by an ERC-721 token. This is how the CAP network knows it is talking to your specific agent and not an impostor.

Setting up an agent on CAP

The steps are:

1. Register on the CROO dashboard. Create an agent identity. This mints your DID.
2. List your services in the dashboard with their names, descriptions, input schemas, and prices.
3. Fund your agent's AA wallet with enough USDC to cover the gas for initial transactions on Base.
4. Add the CROO SDK to your project: `pnpm add @croo-network/sdk`
5. Wire your listener to the SDK's event stream. Accept negotiations, handle paid orders, deliver results.
6. Run `pnpm worker` to start listening.

The critical thing CAP beginners miss: service registration and pricing happen in the dashboard, not in code. The SDK is only for the runtime behavior. Do not look for a "register service" call in the SDK. It is not there.

The listener pattern

```
import { AgentClient, EventType } from "@croo-network/sdk";

const client = new AgentClient(
  { baseUrl: process.env.CROO_BASE_URL, wsURL: process.env.CROO_WS_URL, rpcURL:
process.env.CROO_RPC_URL },
  process.env.CROO_SDK_KEY
);

const stream = await client.connectWebSocket();

stream.on(EventType.NegotiationCreated, async ({ id }) => {
  await client.acceptNegotiation(id);
});

stream.on(EventType.OrderPaid, async ({ id }) => {
  const order = await client.getOrder(id);
  const result = await runService(order.requirements);

  await client.deliverOrder(id, {
    deliverableType: DeliverableType.Schema,
    deliverableSchema: result,
  });
});
```

That is the core pattern. Catch the event, do the work, deliver. The SDK handles the WebSocket reconnect and the on-chain calls.

MOCK mode for development

You do not need a CROO key to develop the agent logic.

In Elfagents, a MOCK switch in `src/lib/config.ts` fires synthetic paid orders locally. The full lifecycle runs: negotiate, accept, pay, work, receipt, deliver. None of it hits the network.

This means you can build and test the entire service logic offline. Switch MOCK off when you are ready to go live.

```
// config.ts
export const MOCK = process.env.CROO_SDK_KEY === undefined;
```

If the SDK key is not set, run in mock mode. If it is set, go live. The rest of the code does not care.

Part 8: Provenance and Trust

Why receipts matter

The problem with agent output is that nobody can trust it.

Any agent can claim it verified a source. Any agent can claim its deliverable conforms to a schema. Claims are cheap.

Receipts are different.

A receipt is a content-addressed record: `keccak256(JSON.stringify(result))`. The hash is computed in the provider (you). The receipt is signed by your agent's wallet. The buyer gets the hash and can verify it has not changed.

In Elfagents, every receipt is also hash-chained to the previous job in the session. If you alter receipt #3, you break the chain from #4 onward. The tampering is visible.

This is the same idea as a blockchain. Each block depends on the previous one. You can verify the whole chain with the public key.

Implementing hash-chained receipts

```
import { keccak256, toHex, toBytes } from "viem";

type Receipt<T> = {
  jobId: string;
  serviceId: string;
  result: T;
  resultHash: string;
  prevHash: string | null;
  chainHash: string;
  timestamp: number;
};

function createReceipt<T>({
  jobId: string,
  serviceId: string,
  result: T,
  prevReceipt: Receipt<unknown> | null
}): Receipt<T> {
  const resultHash = keccak256(toBytes(JSON.stringify(result)));
  const prevHash = prevReceipt?.chainHash ?? null;
  const chainData = `${resultHash}${prevHash ?? ""}`;
  const chainHash = keccak256(toBytes(chainData));

  return {
    jobId,
    serviceId,
    result,
    resultHash,
    prevHash,
    chainHash,
    timestamp: Date.now(),
  };
}
```

Chain every receipt to the one before it. The first receipt in a session has `prevHash: null`. Every subsequent one includes the previous chain hash. If any receipt in the chain is altered, the downstream hashes no longer match.

This is the trust layer that makes agent output auditable. Not infallible, but dramatically better than nothing.

Part 9: Quick Setup Guides

Minimal Claude agent repo

```
mkdir my-agent && cd my-agent

npm init -y

npm install @anthropic-ai/sdk dotenv

touch index.ts .env .gitignore

echo ".env" >> .gitignore

echo "ANTHROPIC_API_KEY=your_key_here" >> .env
```

```

// index.ts

import Anthropic from "@anthropic-ai/sdk";
import "dotenv/config";

const client = new Anthropic();

const tools = [
  {
    name: "get_current_time",
    description: "Returns the current UTC time as an ISO string.",
    input_schema: { type: "object", properties: {}, required: [] },
  },
];

async function executeTool(name: string, _input: any): Promise<string> {
  if (name === "get_current_time") {
    return new Date().toISOString();
  }

  return "Tool not found";
}

async function run() {
  const messages: any[] = [
    { role: "user", content: "What time is it right now?" },
  ];

  while (true) {
    const response = await client.messages.create({
      model: "claude-opus-4-8",
      max_tokens: 1024,
      tools,
      messages,
    });

    messages.push({ role: "assistant", content: response.content });
  }
}

```

```

    if (response.stop_reason === "end_turn") {

        const text = response.content.find((b: any) => b.type === "text");

        console.log(text?.text);

        break;
    }

    if (response.stop_reason === "tool_use") {

        const results = [];

        for (const block of response.content) {

            if (block.type === "tool_use") {

                const result = await executeTool(block.name, block.input);

                results.push({

                    type: "tool_result",

                    tool_use_id: block.id,

                    content: result,

                });

            }

        }

        messages.push({ role: "user", content: results });

    }

}

run().catch(console.error);

```

That is a complete, runnable agent. Clone this shape for any project.

Minimal Gemini agent repo

```
npm install @google/generative-ai dotenv
```

```
import { GoogleGenerativeAI, FunctionCallingMode } from "@google/generative-ai";
import "dotenv/config";

const genAI = new GoogleGenerativeAI(process.env.GEMINI_API_KEY!);

const tools = [
  {
    functionDeclarations: [
      {
        name: "get_current_time",
        description: "Returns the current UTC time",
        parameters: { type: "object", properties: {}, required: [] },
      },
    ],
  },
];

const model = genAI.getGenerativeModel({
  model: "gemini-2.0-flash",
  tools,
  toolConfig: { functionCallingConfig: { mode: FunctionCallingMode.AUTO } },
});

const chat = model.startChat();

const result = await chat.sendMessage("What time is it right now?");
const response = result.response;

const functionCalls = response.functionCalls();
if (functionCalls?.length) {
  const call = functionCalls[0];
  const output = new Date().toISOString();

  const followUp = await chat.sendMessage([
    {

```

```
functionResponse: {  
  name: call.name,  
  response: { result: output },  
},  
},  
]);  
  
console.log(followUp.response.text());  
  
} else {  
  console.log(response.text());  
}
```

Environment checklist

Before running any agent in production:

- ☐ `ANTHROPICAPIKEY` is set and valid (test with a single `messages.create` call)
- ☐ `GEMINIAPIKEY` is set if using Gemini
- ☐ `GITHUB_TOKEN` is set if any tool makes GitHub API calls
- ☐ `.env` is in `.gitignore`
- ☐ No API keys or private keys are hardcoded anywhere in the source
- ☐ Tool handlers return structured error messages on failure
- ☐ Agent loop has a max iteration limit
- ☐ Streaming is used if `max_tokens` is above 16K
- ☐ Rate limit retry logic is in place

Part 10: Where to Go Next

If you are just starting

1. Build the minimal Claude agent above. Get it running locally.
2. Add one real tool. A GitHub search, a web fetch, a database query. Something that touches the outside world.

3. Test the failure modes. What happens when the tool returns an error? What happens when the model calls the tool with wrong arguments?
4. Add a max iteration limit. Test that it fires.
5. Move to streaming.

That sequence alone puts you ahead of most people calling themselves AI engineers.

If you want to ship a paid agent

1. Understand the CAP order lifecycle before writing a single line.
2. Build the agent logic first, with MOCK mode. Do not touch the CAP SDK until the core service works.
3. Register on the CROO dashboard. Get your DID and your AA wallet address.
4. Fund the wallet with enough USDC for gas.
5. Add your SDK key to `.env.local`.
6. Run `pnpm worker` and watch a real negotiation come in.

The hardest part is not the code. It is understanding that your agent is a business: a listed price, a service description, a delivery record that lives on-chain.

Things worth reading

- Anthropic API docs: `docs.anthropic.com`
 - CROO / CAP docs: `docs.croo.network`
 - Google AI Studio: `aistudio.google.com`
 - Elfgents repo (open source): the README and `docs/ARCHITECTURE.md` cover the full service pattern
-

Closing

Building an agent is not hard.

Building an agent that is reliable, auditable, and useful to someone else is hard.

The difference is in the failure modes you planned for, the receipts you generated, the context you managed, and the tools you wrote carefully instead of quickly.

The sense → plan → act → observe loop is the foundation.

Everything else is engineering.

Start simple. Ship something real. Then make it callable.

Anaydike Divine (@sage_senpeak) | Built on Claude Opus 4.8 and the CROO Agent Protocol